

房产 Skills 管理指南



Skills 仓库: https://code.byted.org/life_service/fangchan-cli-repo

Skills Hub 空间:

https://skills.bytedance.net/space/skills.byted.org%2Flife_service%2Ffangchan

一、Skills 概念

如果你已掌握, 可直接跳过本章节

📖 Agent Skill 小白入门蓝皮书

Agent Skill, 是一套标准化的能力封装, 它将复杂的、需要多步推理和工具调用的任务, 打包成一个可以直接使用的"技能包"

一个Agent Skill 的经典结构:

一个 Skill 通常是以一个文件夹的形式存在的, 文件夹的目录结构如下:

代码块

```
1 skill-name/  
2 |— SKILL.md # 必需, < 200 行 (理想)  
3 |   |— YAML frontmatter # name + description (必需)  
4 |   |— Markdown 正文 # 指导内容 (必需)  
5 |— Bundled Resources # 可选  
6 |   |— scripts/ # 可执行脚本  
7 |   |— references/ # 按需加载的文档  
8 |   |— assets/ # 输出使用的文件
```

核心逻辑与三层架构

层级	内容示例	加载时机	作用与价值
L1 元数据	name: code-gen description: 生成符合规范的 TypeScript 代码	常驻 (初始化 / 启动时)	让模型 “知道有什么技能”, 做路由与意图匹配, 占比极低 (~100 token)

L2 核心指令	SKILL.md 中的规则、流程、约束	触发时（匹配意图 / 手动调用）	加载完整执行逻辑，保证任务按规范完成
L3 资源文件	references/ scripts/、examples/ 等	执行中需要时（动态注入）	按需加载辅助资源，避免冗余

当你日常开发中遇到大量的确定性的重复 workflows 时，这个时候就可以沉淀一个 Skills 了。它可以作为一个原子模块完成一个确定性的任务，当你想进阶完成一项复杂任务，也可以通过使用 subagent、其他 skill、MCP、script 等组合能力来完成。

二、背景

随着 AI Coding 的探索与迭代，各业务同学开始沉淀开发场景的最佳实践。然而，在房产业务这种涉及多代码仓库、多技术栈的协作环境中，这些优秀的实践（即“Skills”）往往散落在各处，存在着几个典型痛点：

- **版本不一致与同步难题**：Skills 或存放于本地，或跟随业务仓库发布，缺乏统一管理，导致不同开发者、不同项目间使用的版本不一，能力参差不齐，同步更新困难。
- **“隐身技能”误触发**：为了覆盖所有可能场景，往往会“应安尽安”，在环境中安装大量 Skills。这使得 AI 在面对模糊指令时，容易因关键词匹配而误触发非预期的“隐身技能”，导致执行偏离轨道。
- **缺乏项目级依赖管理**：无法像 `npm` 包那样，按项目维度清晰地声明和锁定其所需的 Skills 组合及其版本。
- **技能质量把控**：缺少技能创建、检测能力，导致在缺乏规范和评估的情况下，技能偏定制化，无法被他人复用。

为了解决这些问题，我们需要统一管理团队通用 skills，并且引入一套类似 `npm` 的项目级 DevDependencies Skills 版本管理机制，通过 `skills-lock.json` 文件来精确控制依赖的安装、更新与版本锁定，确保开发流程的标准化与一致性。

三、统一团队仓库管理

 Skills 仓库：https://code.byted.org/life_service/fangchan-cli-repo

该仓库用于沉淀面向日常开发的脚本化工作流（skills、cli）。

四、工程管理

基于 `skills CLI` 能力做 skills 包的创建、发布、分发、消费、版本锁定、更新。

CLI 命令如下： [🇨🇳 AI 扩展能力市场 QuickStart](#)

Skills 仓库：集中管理 cli + skills

团队建立仓库，做技能集中管理（在该仓库下，技能同步到 master 之后，会自动发布到 skills hub）

代码块

```
1  fangchan-cli-repo/
2  |— packages/                # CLI 目录
3  |   |— d2c-cli
4  |   |— docs-cli
5  |   |— icons-cli
6  |   |— ...
7  |— skills/                  # skills 目录
8  |   |— d2c-diff-workflow
9  |   |— doc-cli
10 |   |— ...
11 |— AGENTS.md
12 |— README.md
13
```

业务仓库：skills-lock.json 依赖声明和锁定

在业务仓库中，通过 `skills` 命令行工具安装技能后，会自动在项目根目录生成一个 `skills-lock.json` 文件。该文件清晰地声明了项目所依赖的 Skills 及其精确的版本信息（通过 `computedHash` 锁定），确保团队所有成员及 AI 使用的都是版本一致的标准化能力。

代码块

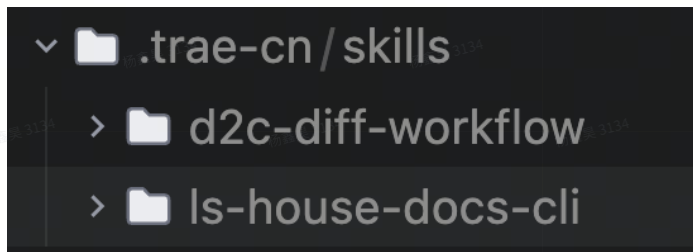
```
1  {
2    "version": 1,
3    "skills": {
4      "ls-house-updating-bam": {
5        "source": "code.byted.org:life_service/fangchan-cli-repo",
6        "sourceType": "git",
7        "computedHash":
8          "0d4c8963eb48e27679ca195d4177d50aa362117a8d7c5c72d91df5a6846a0277"
9      }
10   }
11 }
```

ignore 业务仓库 skills 的提交

与 `node_modules` 类似，通过 `skills` 安装到本地的技能文件夹（默认为 `/.skills/`）是构建产物，**推荐不要提交到 Git 仓库**，以避免潜在的代码冲突和仓库体积异常增大等问题。业务开发者只需关注 `skills-lock.json` 中的版本号即可。

代码块

```
1 .gitignore 文件
2
3 .*/skills
```



安装 skills

当执行项目依赖安装时（如 `emo i`），工程流程会自动检测 `skills-lock.json` 文件，并根据其中锁定的版本同步安装所有必需的 Skills。

```
"scripts": {
  "prepare": "husky install git-hooks",
  "format:ci": "pretty-quick --since HEAD~1",
  "format": "pretty-quick --staged",
  "commit": "cz",
  "setup": "npm install -g pnpm && npm install -g @ies/eden-monorepo@3.9.0",
  "lint": "eslint .",
  "install": "npx skills experimental_install"
},
```

五、如何给业务项目新增 skills

你可以通过 [skills CLI](#) 的命令去从各个源拉取下载。

下载技能

```
1 # 安装内场 skills
2 npm i skills -g --
  registry=https://bnpm.byted.org
3
4 # 从 团队life_service/fangchan-
  cli-repo 里下载 ls-house-updating-
  bam 技能
5 npx skills add
  code.byted.org:life_service/fangc
```

skills.lock.json

```
1 {
2   "version": 1,
3   "skills": {
4     "ls-house-updating-bam": {
5       "source":
6         "code.byted.org:life_service/fang
7         chan-cli-repo",
8       "sourceType": "git",
9       "computedHash":
10        "0d4c8963eb48e27679ca195d4177d50a"
```

```
han-cli-repo --skill ls-house-
updating-bam
6
7 # 从 skills hub 上下载技能
8 skills add
skills.byted.org/life_service/fan
gchan --skill ls-house-updating-
bam --version 1.0.3 -y
```

```
a362117a8d7c5c72d91df5a6846a0277"
8      }
9      }
10     }
11
```

下载完技能后，会在项目根目录下更新 lock 文件

四、如何贡献 Skills

命名规范

团队 Skill 命名规范：

- 要求语义化、清晰、不泛化，一眼就能看懂这个技能是干嘛的
- 统一前缀 `ls-house`，如 `ls-house-updating-bam`
- 使用 **kebab-case** 风格
- 推荐**动名词 (verb-ing) + 名词** 结构，想象“这个技能正在 ____ing 我的 ____”，以遵循单一职责原则，如 `updating-bam`、`finding-okee-pc-comp`
- 长度 <= 64 字符
- 避免模糊、过简的命名，如 `helper`、`utils`、`tools`、`data`、`code`

- ✓
- `ls-house-debugging-react-error`
 - `ls-house-generating-git-commit-message`
 - `ls-house-creating-api-documents`

- ✗
- `debugReact`、`error_debug`
 - `gitCommit`、`commit_msg_gen`
 - `makeApiDoc`、`api_doc_create`

如何开发

用 Skill 创造 Skill

Skill	功能	安装命令
<code>create-skill</code>	Anthropic skill-creator 是官方推出的“元技能”（Meta-Skill）——专门用来创建、测试、优化、打包其他 Claude Skills 的自动化工具链。它把 Skill 开发从纯手工编写	<code>npx skills add</code> https://github.com/siviter-xyz/dot-agent --skill <code>create-skill</code>

YAML/Markdown，变成交互式引导 + 自动生成 + 评估迭代 的工程化流程。

甚至在开发之前会去扫描是否已经有类似能力的 skill，避免重复建设。

```
coco
• 好的！让我来帮你创建一个 skill。
  首先我需要了解一些信息：
  1. 这个 skill 要做什么？ 它应该帮助 Claude 完成什么样的任务？
  2. 什么时候应该触发？ 用户说什么话或在什么场景下应该使用这个 skill？
  3. 期望的输出格式是什么？ 比如生成代码、生成文档、执行特定流程等。
  请告诉我你的想法，我来帮你设计和实现！

Thought for 15s

> 当用户输入 bam update 的时候，去根据 bam.config.js 的配置来拉取 idl

• 好的，让我先了解一下项目中现有的 bam.config.js 和相关 bam 命令的使用方式。
• Explore(探索 bam 相关配置和用法)
  ↳ Done (28 tool uses, 12 KB) (ctrl+o to expand)
• 我发现项目中已经存在一个 ls-house-updating-bam skill，功能和你的描述的基本一致。让我看看它的具体内容。
• Read(file_path: "/Users/bytedance/putong/code/fe_house_mono/.trae/skills/ls-house-updating-bam/evals/evals.json")
  ↳ {
    "skill_name": "bam",
    "evals": [
      ... (+28 line(s))
    ]
  }
• Read(file_path: "/Users/bytedance/putong/code/fe_house_mono/.trae/skills/ls-house-updating-bam/SKILL.md")
  ↳ name: ls-house-updating-bam
  description: 当用户需要更新同步 BAM (APT 管理平台) 接口定义时使用。触发条件包括提到 "bam update"，"更新一下 bam" 或类似包含 "cloud.bytedance.net/bam/rdl/" 的 BAM 链接。这个技能将自动探索项目中的 bam.config.js 位置，解析链接中的 PSM 和版本号，更新配置文件，并在正确目录下执行 bam 命令来拉取最新接口文件。
  ... (+99 line(s))
• 项目中已经存在一个功能完善的 skill: .trae/skills/ls-house-updating-bam/SKILL.md，它已经实现了你描述的功能：
  已有功能：
  1. 当用户输入 "bam update" 时，自动探索 bam.config.js 位置并执行 npm bam update
  2. 当用户提供 BAM 链接时，自动解析 PSM 版本号/分支名，更新配置后执行
  3. 支持多种配置风格 (简单字符串 / 对象值) 的自适应
  4. 多个 bam.config.js 时会让用户选择
  这个 skill 已经在 available_skills 中注册为 ls-house-updating-bam。
  你是想：
  1. 直接使用它 (输入 "bam update" 即可触发)
  2. 改进现有 skill (比如增加新功能、修复问题)
  3. 创建一个全新的、不同的 skill
  请告诉我你的需求，我来帮你处理！
```

示例：

1. 你只需要说帮我创建一个 skill，就能触发交互式引导，一步一步完成开发了

```
> 帮我创建一个 skill
  ↳ Read .trae/rules/project_rules.md (42 lines)
  ↳ Read AGENTS.md (353 lines)
  ↳ Read ../../../../AGENTS.md (3 lines)

• Skill(skill: "skill-creator")
  ↳ skill-creator's instructions have been read.

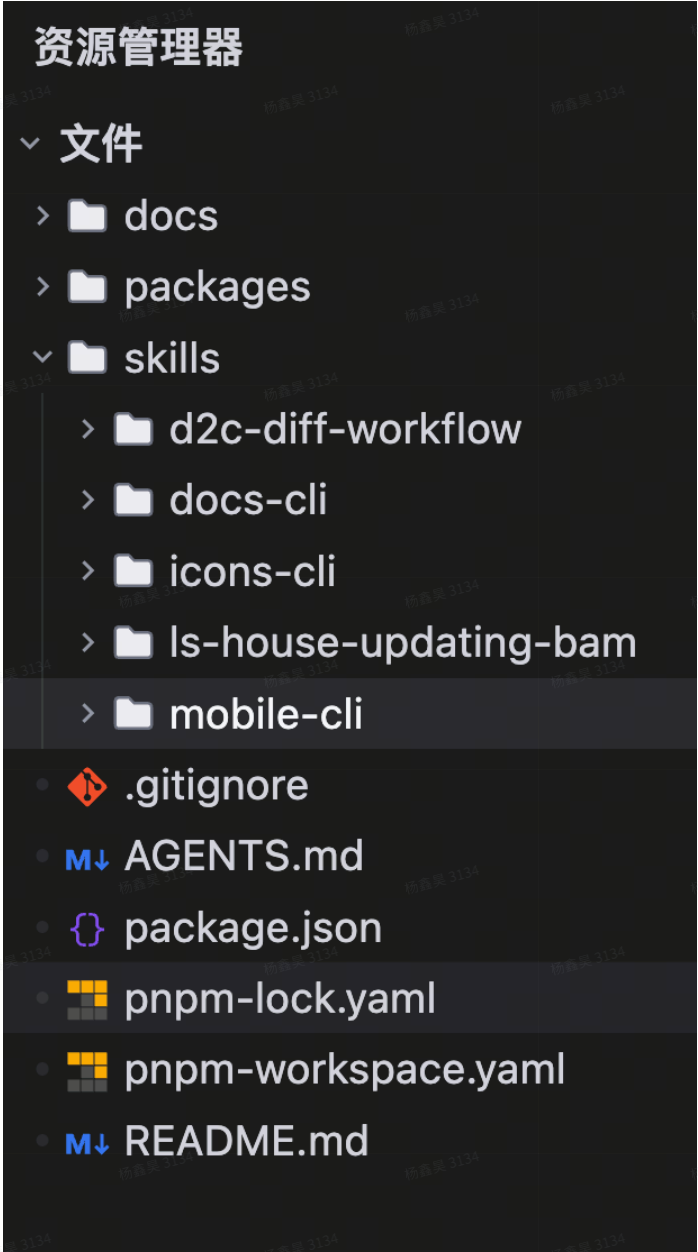
• 好的！让我来帮你创建一个 skill。
  首先我需要了解一些信息：
  1. 这个 skill 要做什么？ 它应该帮助 Claude 完成什么样的任务？
  2. 什么时候应该触发？ 用户说什么话或在什么场景下应该使用这个 skill？
  3. 期望的输出格式是什么？ 比如生成代码、生成文档、执行特定流程等。
  请告诉我你的想法，我来帮你设计和实现！

Thought for 15s

> 当用户输入 bam update 的时候，去根据 bam.config.js 的配置来拉取 idl

• 好的，让我先了解一下项目中现有的 bam.config.js 和相关 bam 命令的使用方式。
* Thinking... (10s  ↳ 142 B)
```

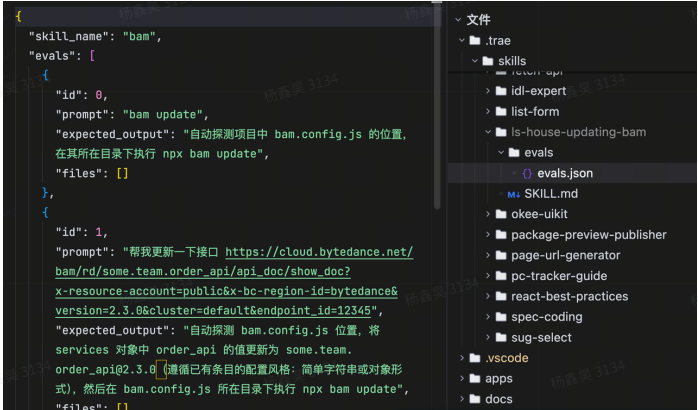
2. 完成测试开发之后，直接提交到 skills 目录下即可，仓库配置了 trigger，可以自动触发向 hub 的更新同步



质检

//Todo wip

虽然 create-skill 的机制生成的 skills 已经是一个质量比较高的 skills 了，比如它甚至会帮助你写测试用例，以保证 skills 的稳定性



但是作为整个团队通用的 skills，它还得具备通用性，否则基本上就只能专项专用、专人专用。比如不能写死路径、比如不能写死输出格式

	Before	After
bam.config.js s 配置路径写		

